UTS

PEMBELAJARAN MESIN



NAMA : Rendy Pradika Abiyoga

NIM : 202431111

KELAS : G

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : -

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Morgan Immanuel Nainggolan

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I PENDAHULUAN 4
1.1 Rumusan Masalah 4
1.2 Tujuan Masalah 4
1.3 Manfaat Masalah 4
BAB II 5
LANDASAN TEORI 5
2.1 Machine Learning 5
2.2 Supervised Learning 5
2.3 Klasifikasi Data 5
2.4 Exploratory Data Analysis (EDA) 6
2.5 Data Preprocessing 7
2.6 Algoritma Naive Bayes 9
2.6.1 Teorema Bayes 10
2.6.2 Prinsip Kerja Naive Bayes 10
2.6.3 Gaussian Naive Bayes 10
2.6.4 Kelebihan dan Kekurangan Naive Bayes 11
2.7 Algoritma K-Nearest Neighbors (KNN) 11
2.7.1 Prinsip Kerja KNN 12
2.7.2 Euclidean Distance 12
2.7.3 Pemilihan Nilai K 12
2.7.4 Pengaruh Normalisasi terhadap KNN 13
2.7.5 Kelebihan dan Kekurangan KNN 13
2.8 Evaluasi Model Klasifikasi 13
2.8.1 Confusion Matrix 14
2.8.2 Accuracy 14
2.8.3 Precision 14
2.8.4 Recall 15
2.8.5 F1-Score 15
2.9 Perbandingan Algoritma Naive Bayes dan KNN 15
BAB III HASIL 17

Cell pertama – Import Library dan Membaca Dataset 17
3.1 Persiapan Dataset 17
Cell 2 – Menampilkan ukuran dataset 17
Cell 3 – Menampilkan informasi dataset 18
Cell 4 – Menampilkan statistik deskriptif 18
3.2 Exploratory Data Analysis (EDA) 19
Cell 5 – Memeriksa Missing Values 19
Cell 6 – Memeriksa Data Duplikat 19
Cell 7 – Menganalisis Distribusi Target 20
Cell 8 – Visualisasi Distribusi Target 20
Cell 9 – Visualisasi Distribusi Atribut Numerik 21
Cell 10 – Analisis Korelasi Atribut Numerik 22
Cell 11 – Memeriksa Outliers 23
3.3 Data Preprocessing 23
Cell 12 – Menentukan Atribut Numerik dan Kategorikal 23
Cell 14 – Memisahkan Fitur dan Target 25
Cell 15 – Membagi Data Training dan Testing 26
Cell 16 – Encoding dan Normalisasi Data 27
3.4 Pemodelan menggunakan Naive Bayes 27
Cell 17 – Import library dan membuat model 27
Cell 18 – Melakukan prediksi 28
3.5 Pemodelan menggunakan K-Nearest Neighbors (KNN) 28
Cell 19 – Import library dan membuat model KNN 28
Cell 20 – Melakukan prediksi KNN 29
3.6 Evaluasi model 29
Cell 21 – Evaluasi Naive Bayes 29
Cell 22 – Evaluasi KNN 30
Cell 23 – Membuat tabel perbandingan 31
BAB IV PENUTUP 32
4.1 Kesimpulan 32
DAFTAR PUSTAKA 34

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana memahami karakteristik dan struktur dataset yang digunakan melalui proses *Exploratory Data Analysis* (EDA)?
2. Bagaimana melakukan preprocessing data agar dapat digunakan dalam proses pelatihan model *Machine Learning*?
3. Bagaimana menerapkan algoritma *Naive Bayes* dan *K-Nearest Neighbors* (KNN) untuk melakukan klasifikasi berdasarkan dataset yang diberikan?
4. Bagaimana mengetahui tingkat akurasi model yang dihasilkan setelah proses pelatihan dan pengujian data?
5. Bagaimana menganalisis hasil klasifikasi berdasarkan teori dan hasil pengolahan data yang telah dilakukan?

1.2 Tujuan Masalah

1. Memahami karakteristik dataset melalui proses *Exploratory Data Analysis* (EDA), termasuk struktur data, tipe atribut, distribusi data, dan kemungkinan adanya nilai yang hilang.
2. Melakukan preprocessing data untuk mempersiapkan dataset sebelum digunakan dalam proses pemodelan.
3. Mengimplementasikan algoritma *Naive Bayes* dan *K-Nearest Neighbors* (KNN) dalam proses klasifikasi data.
4. Mengetahui dan memperoleh nilai akurasi dari model yang telah dilatih dan diuji.
5. Menganalisis hasil pengolahan data serta memahami kinerja algoritma berdasarkan teori pembelajaran mesin dan hasil evaluasi model.

1.3 Manfaat Masalah

1. **Menambah Pemahaman Teoritis**: Memberikan pemahaman mengenai konsep dasar *Machine Learning*, khususnya algoritma *Naive Bayes* dan *K-Nearest Neighbors* (KNN), beserta prinsip kerja masing-masing algoritma dalam melakukan klasifikasi.
2. **Meningkatkan Kemampuan Pengolahan Data**: Melatih kemampuan dalam memahami karakteristik dataset, melakukan EDA, serta menerapkan preprocessing untuk mempersiapkan data sebelum digunakan dalam pemodelan.
3. **Mengembangkan Keterampilan Implementasi**: Meningkatkan kemampuan dalam mengimplementasikan algoritma pembelajaran mesin menggunakan bahasa pemrograman Python dan berbagai library pendukung.
4. **Memahami Evaluasi Model**: Memberikan pengalaman dalam melakukan pelatihan dan pengujian model serta memahami penggunaan nilai akurasi untuk mengukur kinerja algoritma klasifikasi.
5. **Melatih Kemampuan Analisis**: Mengembangkan kemampuan dalam menganalisis hasil pengolahan data, mengidentifikasi faktor yang memengaruhi performa model, serta menghubungkan hasil praktikum dengan teori yang telah dipelajari.

BAB II
LANDASAN TEORI

2.1 Machine Learning

Machine Learning merupakan salah satu cabang dari kecerdasan buatan (*Artificial Intelligence*) yang memungkinkan komputer mempelajari pola dan hubungan dari sekumpulan data untuk menghasilkan prediksi atau keputusan tanpa harus diprogram secara eksplisit untuk setiap permasalahan. Dalam penerapannya, algoritma *Machine Learning* memanfaatkan data sebagai dasar pembelajaran sehingga sistem dapat mengenali pola yang terdapat pada data dan menggunakannya untuk menyelesaikan permasalahan tertentu.

Pada proses pembelajaran mesin, kualitas data menjadi salah satu faktor yang memengaruhi kemampuan model dalam menghasilkan prediksi. Oleh karena itu, sebelum model dilatih, data perlu melalui tahapan pemeriksaan, pembersihan, dan pengolahan agar dapat digunakan secara optimal. Penelitian Alshdaifat et al. (2021) menunjukkan bahwa pemilihan teknik preprocessing dapat memberikan pengaruh yang berbeda terhadap performa algoritma klasifikasi.

Dalam praktikum ini, konsep *Machine Learning* diterapkan untuk mempelajari proses pengolahan dataset dan membangun model klasifikasi menggunakan algoritma yang telah ditentukan, yaitu *Naive Bayes* dan *K-Nearest Neighbors* (KNN).

2.2 Supervised Learning

Supervised Learning merupakan metode pembelajaran mesin yang menggunakan data berlabel sebagai bahan pembelajaran. Data tersebut terdiri atas variabel input (*features*) dan variabel target (*label*) yang menjadi hasil atau kategori yang ingin diprediksi oleh model.

Dalam metode ini, algoritma mempelajari hubungan antara variabel input dan target melalui data pelatihan (*training data*). Setelah proses pembelajaran selesai, model diuji menggunakan data yang belum pernah digunakan dalam pelatihan untuk mengetahui kemampuannya dalam melakukan prediksi terhadap data baru.

Salah satu penerapan *Supervised Learning* adalah klasifikasi, yaitu proses mengelompokkan data ke dalam kategori tertentu berdasarkan pola yang telah dipelajari. Algoritma *Naive Bayes* dan *K-Nearest Neighbors* termasuk metode yang dapat digunakan untuk menyelesaikan permasalahan klasifikasi.

2.3 Klasifikasi Data

Klasifikasi merupakan salah satu teknik dalam *Machine Learning* yang bertujuan untuk menentukan kategori atau kelas dari suatu data berdasarkan karakteristik atribut yang dimilikinya. Proses klasifikasi dilakukan dengan mempelajari pola dari data yang telah memiliki label, kemudian menggunakan pola tersebut untuk menentukan kelas dari data baru.

Dalam proses klasifikasi, terdapat dua komponen utama, yaitu:

1. **Features (Atribut):** Variabel atau karakteristik yang digunakan sebagai dasar dalam proses pembelajaran dan prediksi.
2. **Target (Label):** Variabel yang menjadi hasil prediksi atau kategori yang ingin ditentukan oleh model.

Sebagai contoh, pada dataset penyakit jantung, atribut dapat berupa usia, tekanan darah, kadar kolesterol, dan karakteristik kesehatan lainnya. Sementara itu, target dapat berupa kategori seseorang mengalami penyakit jantung atau tidak.

Keberhasilan proses klasifikasi dapat diketahui melalui evaluasi hasil prediksi terhadap label sebenarnya. Oleh karena itu, proses pengujian model diperlukan untuk mengetahui kemampuan algoritma dalam mengklasifikasikan data.

2.4 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) merupakan tahapan analisis awal yang dilakukan untuk memahami karakteristik, struktur, distribusi, dan hubungan antarvariabel dalam suatu dataset sebelum dilakukan pemodelan. EDA bertujuan untuk memperoleh gambaran menyeluruh mengenai kondisi data sehingga permasalahan yang terdapat di dalam dataset dapat diketahui sejak awal.

Melalui EDA, peneliti dapat mengidentifikasi nilai yang hilang (*missing values*), data duplikat, distribusi atribut, nilai ekstrem (*outliers*), serta ketidakseimbangan jumlah data pada setiap kelas. Informasi tersebut dapat digunakan sebagai dasar dalam menentukan teknik preprocessing yang sesuai.

Beberapa aspek yang dapat dianalisis melalui EDA antara lain:

1. Pemeriksaan Struktur Dataset

Pemeriksaan struktur dataset dilakukan untuk mengetahui jumlah baris dan kolom, nama atribut, tipe data, serta gambaran umum informasi yang terdapat di dalam dataset. Tahapan ini membantu menentukan jenis pengolahan yang diperlukan untuk setiap atribut.

2. Analisis Statistik Deskriptif

Statistik deskriptif digunakan untuk mengetahui karakteristik data numerik melalui nilai rata-rata (*mean*), nilai tengah (*median*), standar deviasi, nilai minimum, dan maksimum. Informasi tersebut dapat membantu memahami penyebaran serta kecenderungan nilai pada setiap atribut.

3. Analisis Distribusi Data

Analisis distribusi dilakukan untuk mengetahui pola penyebaran nilai dalam suatu atribut. Visualisasi seperti histogram dan diagram batang dapat digunakan untuk melihat frekuensi kemunculan nilai serta mengetahui apakah distribusi data cenderung merata atau tidak.

4. Analisis Korelasi

Analisis korelasi digunakan untuk mengetahui hubungan antara dua atau lebih variabel numerik. Nilai korelasi dapat menunjukkan arah dan tingkat hubungan antarvariabel. Analisis ini dapat membantu memahami keterkaitan atribut dalam dataset.

5. Identifikasi Missing Values dan Outliers

Missing values merupakan kondisi ketika suatu atribut tidak memiliki nilai atau informasi yang lengkap. Sementara itu, *outliers* merupakan data yang memiliki nilai jauh berbeda dibandingkan sebagian besar pengamatan lainnya. Kedua kondisi tersebut perlu diperiksa karena dapat memengaruhi proses analisis dan performa model.

EDA menjadi tahapan penting karena membantu memastikan bahwa karakteristik dataset telah dipahami sebelum proses pengolahan dan pemodelan dilakukan.

2.5 Data Preprocessing

Data preprocessing merupakan proses mempersiapkan dan mengolah data mentah agar memiliki format serta kualitas yang sesuai untuk digunakan dalam algoritma *Machine Learning*. Data yang diperoleh dari berbagai sumber tidak selalu memiliki struktur yang konsisten dan dapat mengandung nilai kosong, data duplikat, perbedaan skala, maupun format atribut yang tidak sesuai.

Menurut Alshdaifat et al. (2021), pemilihan teknik preprocessing dapat memberikan pengaruh terhadap performa algoritma klasifikasi. Oleh karena itu, proses preprocessing perlu disesuaikan dengan karakteristik dataset dan kebutuhan algoritma yang digunakan.

Beberapa tahapan preprocessing yang umum dilakukan adalah sebagai berikut:

1. Data Cleaning

Data cleaning merupakan proses membersihkan dataset dari permasalahan yang dapat mengganggu proses analisis. Proses ini dapat meliputi pemeriksaan data duplikat, nilai kosong, kesalahan format, dan ketidaksesuaian tipe data.

2. Missing Value Handling

Penanganan *missing values* dilakukan untuk mengatasi atribut yang memiliki nilai kosong. Beberapa metode yang dapat digunakan adalah:

- Menghapus baris atau kolom yang memiliki nilai kosong apabila jumlahnya tidak signifikan atau atributnya tidak diperlukan.
- Mengisi nilai kosong menggunakan rata-rata (*mean*) untuk data numerik tertentu.
- Mengisi nilai kosong menggunakan median apabila data memiliki distribusi yang tidak simetris atau mengandung nilai ekstrem.
- Mengisi nilai kosong menggunakan modus (*mode*) untuk data kategorikal.

Penelitian Palanivinayagam dan Damaševičius (2023) membahas bahwa keberadaan nilai yang hilang dapat mengurangi informasi yang dipelajari model dan berpotensi memengaruhi akurasi klasifikasi.

3. Encoding Data

Encoding merupakan proses mengubah data kategorikal menjadi representasi numerik agar dapat diproses oleh algoritma pembelajaran mesin yang membutuhkan input numerik.

Salah satu metode yang umum digunakan adalah *Label Encoding*, yaitu memberikan representasi angka pada setiap kategori. Selain itu, terdapat *One-Hot Encoding* yang mengubah kategori menjadi beberapa kolom biner.

Pemilihan metode encoding harus mempertimbangkan jenis atribut agar tidak menghasilkan representasi data yang menyesatkan, terutama pada atribut kategorikal yang tidak memiliki urutan.

4. Normalization dan Standardization

Normalisasi dan standardisasi merupakan teknik untuk menyamakan skala nilai antaratribut numerik. Proses ini sangat penting bagi algoritma yang menggunakan perhitungan jarak, seperti KNN.

Dua metode yang umum digunakan adalah:

- **Min-Max Normalization**

  Teknik ini mengubah nilai data ke dalam rentang tertentu, biasanya antara 0 sampai 1.

  Rumus:

$$X_{\text{norm}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

  Keterangan:

  - (X') = nilai setelah normalisasi.
  - (X) = nilai awal.
  - (X_{min}) = nilai minimum.
  - (X_{max}) = nilai maksimum.

- **Standardization (Z-Score)**

  Teknik ini mengubah data berdasarkan nilai rata-rata dan standar deviasi sehingga distribusi hasil transformasi memiliki rata-rata 0 dan standar deviasi 1.

Rumus:

$$Z = \frac{X - \mu}{\sigma}$$

Keterangan:

- (Z) = nilai hasil standardisasi.
- (X) = nilai awal.
- (miu) = nilai rata-rata.
- (sigma) = standar deviasi.

Penelitian Izonin et al. (2022) menjelaskan bahwa normalisasi merupakan salah satu tahapan preprocessing yang dapat membantu meningkatkan kualitas pengolahan data dalam permasalahan klasifikasi.

5. Data Splitting

Data splitting merupakan proses membagi dataset menjadi dua bagian utama, yaitu data pelatihan (*training data*) dan data pengujian (*testing data*).

- **Training Data:** Digunakan untuk melatih model agar dapat mempelajari pola dari dataset.
- **Testing Data:** Digunakan untuk menguji kemampuan model dalam melakukan prediksi terhadap data yang belum digunakan selama pelatihan.

Pembagian data dapat menggunakan proporsi 80:20, yaitu 80% untuk data pelatihan dan 20% untuk data pengujian. Proporsi tersebut merupakan salah satu pilihan yang umum digunakan dan perlu disesuaikan dengan jumlah serta karakteristik dataset.

Pada proses pembagian data, metode *stratified sampling* dapat digunakan untuk mempertahankan proporsi kelas pada data pelatihan dan pengujian, terutama apabila distribusi kelas tidak seimbang.

Selain itu, proses preprocessing yang mempelajari parameter dari data, seperti normalisasi dan imputasi, perlu dilakukan dengan memperhatikan pemisahan data. Parameter preprocessing sebaiknya dipelajari dari data pelatihan terlebih dahulu, kemudian diterapkan pada data pengujian untuk menghindari *data leakage*.

2.6 Algoritma Naive Bayes

Naive Bayes merupakan algoritma klasifikasi yang menggunakan prinsip probabilitas dan Teorema Bayes untuk menentukan kemungkinan suatu data termasuk ke dalam kelas tertentu. Algoritma ini memperkirakan kelas berdasarkan probabilitas setiap atribut terhadap kelas yang diprediksi.

Nama *Naive* berasal dari asumsi bahwa setiap atribut memiliki sifat independen secara kondisional terhadap kelas yang ditentukan. Artinya, algoritma menganggap bahwa hubungan suatu atribut dengan kelas tidak bergantung pada atribut lainnya setelah kelas diketahui.

Menurut Alenazi et al. (2023), salah satu karakteristik utama Naive Bayes adalah penggunaan asumsi independensi kondisional antaratribut. Pelanggaran terhadap asumsi tersebut dapat memengaruhi performa klasifikasi.

2.6.1 Teorema Bayes

Dasar perhitungan algoritma Naive Bayes adalah Teorema Bayes yang digunakan untuk menghitung probabilitas suatu kelas berdasarkan informasi atribut yang tersedia.

1. Rumus Teorema Bayes

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Keterangan:

- (P(C|X)) = probabilitas kelas (C) berdasarkan data (X).
- (P(X|C)) = probabilitas data (X) jika diketahui termasuk kelas (C).
- (P(C)) = probabilitas awal kelas (*prior probability*).
- (P(X)) = probabilitas kemunculan data (X).

Dalam proses klasifikasi, algoritma menghitung probabilitas setiap kelas dan memilih kelas dengan nilai probabilitas posterior tertinggi.

2.6.2 Prinsip Kerja Naive Bayes

Secara umum, proses kerja Naive Bayes terdiri atas beberapa tahapan:

1. Mengidentifikasi kelas target yang terdapat dalam dataset.
2. Menghitung probabilitas awal (*prior probability*) dari setiap kelas.
3. Menghitung probabilitas setiap atribut terhadap masing-masing kelas.
4. Menggabungkan probabilitas atribut berdasarkan asumsi independensi kondisional.
5. Membandingkan probabilitas posterior dari setiap kelas.
6. Menentukan kelas dengan probabilitas tertinggi sebagai hasil klasifikasi.

2.6.3 Gaussian Naive Bayes

Gaussian Naive Bayes merupakan salah satu jenis Naive Bayes yang digunakan untuk menangani atribut numerik dengan asumsi bahwa nilai atribut mengikuti distribusi normal atau Gaussian.

Rumus fungsi probabilitas Gaussian:

$$P(x_i|C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right)$$

Keterangan:

- (xi) = nilai atribut yang diamati.
- (miuC) = rata-rata atribut pada kelas (C).
- (sigmaC^2) = varians atribut pada kelas (C).
- (phi) = konstanta matematika.
- (e) = bilangan eksponensial.

Gaussian Naive Bayes memperkirakan probabilitas suatu nilai numerik berdasarkan distribusi normal pada setiap kelas.

2.6.4 Kelebihan dan Kekurangan Naive Bayes

Kelebihan algoritma Naive Bayes:

1. Memiliki proses pembelajaran yang relatif sederhana.
2. Memiliki kebutuhan komputasi yang relatif rendah.
3. Dapat digunakan untuk klasifikasi dengan jumlah data yang cukup besar.
4. Menghasilkan prediksi berdasarkan perhitungan probabilitas.
5. Dapat bekerja dengan baik pada berbagai permasalahan klasifikasi.

Kekurangan algoritma Naive Bayes:

1. Memiliki asumsi independensi kondisional antaratribut yang tidak selalu sesuai dengan kondisi data sebenarnya.
2. Performa dapat menurun ketika terdapat hubungan yang kuat antaratribut.
3. Pada Gaussian Naive Bayes, asumsi distribusi normal dapat memengaruhi ketepatan estimasi probabilitas apabila karakteristik data tidak sesuai.
4. Estimasi probabilitas dapat mengalami kendala ketika terdapat data yang sangat sedikit pada kategori tertentu.

2.7 Algoritma K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) merupakan algoritma pembelajaran mesin yang digunakan untuk klasifikasi berdasarkan kedekatan jarak antara data yang akan diprediksi dengan data yang telah diketahui kelasnya.

Algoritma KNN bekerja dengan mencari sejumlah (K) tetangga terdekat dari suatu data berdasarkan perhitungan jarak. Kelas yang paling banyak muncul di antara tetangga terdekat tersebut akan digunakan sebagai hasil prediksi.

Menurut Zhang et al. (2022), KNN merupakan metode klasifikasi yang sederhana dan banyak digunakan, tetapi performanya dipengaruhi oleh pemilihan jumlah tetangga dan metode perhitungan jarak.

2.7.1 Prinsip Kerja KNN

Secara umum, algoritma KNN memiliki tahapan kerja sebagai berikut:

1. Menentukan nilai (K) sebagai jumlah tetangga terdekat yang akan digunakan.
2. Menghitung jarak antara data pengujian dengan seluruh data pelatihan.
3. Mengurutkan hasil perhitungan jarak dari yang terkecil hingga terbesar.
4. Mengambil sebanyak (K) data dengan jarak terdekat.
5. Menghitung jumlah kemunculan kelas dari tetangga yang terpilih.
6. Menentukan kelas dengan jumlah suara terbanyak sebagai hasil prediksi.

2.7.2 Euclidean Distance

Euclidean Distance merupakan metode pengukuran jarak yang digunakan untuk menghitung jarak antara dua titik berdasarkan atribut numerik yang dimiliki.

Rumus Euclidean Distance:

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Keterangan:

- (d(x,y)) = jarak antara dua data.
- (x_i) = nilai atribut pada data pertama.
- (y_i) = nilai atribut pada data kedua.
- (n) = jumlah atribut yang digunakan.

Semakin kecil nilai jarak yang dihasilkan, semakin dekat hubungan antara kedua data tersebut berdasarkan atribut yang dihitung.

2.7.3 Pemilihan Nilai K

Nilai (K) menunjukkan jumlah tetangga terdekat yang digunakan dalam proses klasifikasi. Pemilihan nilai (K) dapat memengaruhi hasil prediksi yang dihasilkan oleh model.

Apabila nilai (K) terlalu kecil, model lebih sensitif terhadap noise atau data yang menyimpang. Sebaliknya, apabila nilai (K) terlalu besar, model dapat kehilangan informasi mengenai pola lokal dari data.

Oleh karena itu, nilai (K) perlu dipilih dengan mempertimbangkan karakteristik dataset dan hasil evaluasi model.

2.7.4 Pengaruh Normalisasi terhadap KNN

Normalisasi merupakan salah satu tahapan penting dalam penerapan KNN karena algoritma ini menggunakan perhitungan jarak untuk menentukan kedekatan antar-data.

Apabila atribut memiliki rentang nilai yang berbeda secara signifikan, atribut dengan skala lebih besar dapat memberikan pengaruh yang lebih dominan terhadap hasil perhitungan jarak.

Penelitian Yusran et al. (2025) membahas pengaruh teknik normalisasi dan pemilihan metrik jarak terhadap performa KNN dalam klasifikasi penyakit diabetes. Hal tersebut menunjukkan bahwa pemilihan metode pengolahan data perlu diperhatikan dalam penerapan algoritma berbasis jarak.

2.7.5 Kelebihan dan Kekurangan KNN

Kelebihan algoritma KNN:

1. Memiliki konsep dan mekanisme kerja yang relatif sederhana.
2. Tidak memerlukan proses pembentukan model yang kompleks.
3. Dapat digunakan untuk permasalahan klasifikasi dengan berbagai jenis dataset yang telah dipersiapkan.
4. Dapat menangani pola klasifikasi yang tidak selalu berbentuk linear.

Kekurangan algoritma KNN:

1. Membutuhkan perhitungan jarak terhadap data pelatihan ketika melakukan prediksi.
2. Performa sangat dipengaruhi oleh pemilihan nilai (K).
3. Sensitif terhadap perbedaan skala antaratribut.
4. Dapat mengalami penurunan performa ketika dataset memiliki banyak atribut yang tidak relevan.
5. Membutuhkan penyimpanan data pelatihan untuk proses klasifikasi.

2.8 Evaluasi Model Klasifikasi

Evaluasi model merupakan tahapan yang dilakukan untuk mengetahui kemampuan algoritma dalam menghasilkan prediksi yang sesuai dengan kelas sebenarnya. Evaluasi dilakukan dengan membandingkan hasil prediksi model terhadap label aktual pada data pengujian.

Salah satu metode yang digunakan untuk mengevaluasi model klasifikasi adalah *Confusion Matrix*. Metode ini menyajikan perbandingan antara hasil prediksi dan kondisi sebenarnya sehingga kesalahan maupun keberhasilan klasifikasi dapat dianalisis.

Menurut Cabot dan Ross (2023), evaluasi model klasifikasi perlu mempertimbangkan beberapa metrik, seperti akurasi, precision, recall, dan F1-score agar performa model dapat dipahami secara lebih menyeluruh.

2.8.1 Confusion Matrix

Pada klasifikasi biner, *Confusion Matrix* terdiri atas empat komponen utama:

1. **True Positive (TP):** Data positif yang berhasil diprediksi sebagai positif.
2. **True Negative (TN):** Data negatif yang berhasil diprediksi sebagai negatif.
3. **False Positive (FP):** Data negatif yang salah diprediksi sebagai positif.
4. **False Negative (FN):** Data positif yang salah diprediksi sebagai negatif.

Keempat komponen tersebut digunakan sebagai dasar perhitungan berbagai metrik evaluasi klasifikasi.

2.8.2 Accuracy

Accuracy merupakan metrik yang digunakan untuk mengetahui persentase jumlah prediksi yang benar dibandingkan dengan seluruh data pengujian.

Rumus:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

Nilai akurasi yang semakin tinggi menunjukkan bahwa semakin banyak data yang berhasil diklasifikasikan dengan benar. Namun, akurasi perlu dianalisis bersama metrik lainnya apabila dataset memiliki distribusi kelas yang tidak seimbang.

2.8.3 Precision

Precision digunakan untuk mengukur seberapa tepat model ketika memprediksi suatu data sebagai kelas positif.

Rumus:

$$Precision = \frac{TP}{TP + FP} \times 100\%$$

Nilai precision yang tinggi menunjukkan bahwa sebagian besar prediksi positif yang dihasilkan model memang benar-benar positif.

2.8.4 Recall

Recall merupakan metrik yang digunakan untuk mengukur kemampuan model dalam menemukan seluruh data yang sebenarnya termasuk kelas positif.

Rumus:

$$Recall = \frac{TP}{TP + FN} \times 100\%$$

Nilai recall yang tinggi menunjukkan bahwa model mampu mengenali sebagian besar data positif yang terdapat dalam dataset.

2.8.5 F1-Score

F1-score merupakan nilai yang menggabungkan precision dan recall menggunakan rata-rata harmonik. Metrik ini digunakan untuk mengetahui keseimbangan antara ketepatan prediksi positif dan kemampuan model dalam menemukan data positif.

Rumus:

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1-score sangat berguna ketika evaluasi model perlu mempertimbangkan keseimbangan antara precision dan recall, terutama pada dataset yang memiliki distribusi kelas tidak seimbang.

2.9 Perbandingan Algoritma Naive Bayes dan KNN

Naive Bayes dan KNN merupakan dua algoritma klasifikasi yang memiliki prinsip kerja berbeda. Naive Bayes menggunakan pendekatan probabilitas berdasarkan Teorema Bayes, sedangkan KNN menggunakan kedekatan jarak antar-data untuk menentukan kelas suatu objek.

Perbedaan kedua algoritma tersebut dapat dilihat berdasarkan beberapa aspek berikut:

Aspek	**Naive Bayes**	**KNN**
Dasar metode	Probabilitas	Perhitungan jarak
Prinsip kerja	Teorema Bayes	Tetangga terdekat
Proses pembelajaran	Mengestimasi probabilitas	Menyimpan data pelatihan

Parameter penting	Distribusi dan asumsi probabilitas	Nilai K dan metrik jarak
Pengaruh skala data	Bergantung pada varian dan representasi data	Sangat sensitif terhadap skala
Proses prediksi	Menghitung probabilitas setiap kelas	Mencari tetangga terdekat
Penentuan kelas	Probabilitas posterior tertinggi	Mayoritas kelas tetangga

Perbedaan prinsip kerja tersebut menyebabkan kedua algoritma dapat menghasilkan performa yang berbeda ketika diterapkan pada dataset yang sama. Oleh karena itu, proses pengujian dan evaluasi diperlukan untuk mengetahui kemampuan masing-masing algoritma berdasarkan karakteristik data yang digunakan.

BAB III

HASIL

Cell pertama – Import Library dan Membaca Dataset

In [1]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Import library
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Membaca dataset
df = pd.read_csv('04_bank_customers.csv', sep=';')

# Menampilkan 5 data pertama
df.head()
```

Out[1]:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no

Cell pertama digunakan untuk mengimpor beberapa library Python yang diperlukan dalam proses pengolahan dan analisis data. Library Pandas digunakan untuk membaca dan mengelola dataset, NumPy digunakan untuk operasi numerik, Matplotlib digunakan untuk membuat visualisasi grafik, sedangkan Seaborn digunakan untuk membantu menampilkan visualisasi data secara lebih informatif.

Selanjutnya, dataset Bank Customers dibaca menggunakan fungsi read_csv() dengan parameter sep=';' karena file menggunakan tanda titik koma sebagai pemisah antar kolom. Data kemudian disimpan ke dalam variabel df dan ditampilkan menggunakan fungsi head() untuk melihat lima baris pertama.

3.1 Persiapan Dataset

Cell 2 – Menampilkan ukuran dataset

In [2]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Menampilkan jumlah baris dan kolom
print("Jumlah Baris dan Kolom:", df.shape)

# Menampilkan jumlah baris
print("Jumlah Data:", df.shape[0])

# Menampilkan jumlah atribut
print("Jumlah Atribut:", df.shape[1])
```

```
Jumlah Baris dan Kolom: (45211, 17)
Jumlah Data: 45211
Jumlah Atribut: 17
```

Cell ini digunakan untuk mengetahui ukuran dataset yang digunakan dalam praktikum. Fungsi shape menghasilkan informasi mengenai jumlah baris dan kolom pada dataset. Berdasarkan hasil pemeriksaan, dataset Bank Customers memiliki 45.211 baris data dan 17 kolom, yang terdiri atas atribut karakteristik nasabah dan satu atribut target untuk proses klasifikasi.

Cell 3 – Menampilkan informasi dataset

```
In [3]: #202431111_RendyPradikaAbiyoga
        # NIM : 202431111
        # Nama : Rendy Pradika Abiyoga

        # Menampilkan informasi struktur dataset
        df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

Cell ini digunakan untuk melihat informasi struktur dataset secara keseluruhan melalui fungsi info(). Informasi yang ditampilkan meliputi nama kolom, jumlah data non-null, dan tipe data setiap atribut. Hasil pemeriksaan menunjukkan bahwa dataset memiliki 7 atribut bertipe numerik (int64) dan 10 atribut bertipe kategorikal (object). Seluruh 17 atribut memiliki 45.211 data non-null.

Cell 4 – Menampilkan statistik deskriptif

```
In [4]: #202431111_RendyPradikaAbiyoga
        # NIM : 202431111
        # Nama : Rendy Pradika Abiyoga

        # Menampilkan statistik deskriptif
        df.describe()
```

Out[4]:

	age	balance	day	duration	campaign	pdays	previous
count	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000
mean	40.936210	1362.272058	15.806419	258.163080	2.763841	40.197828	0.580323
std	10.618762	3044.765829	8.322476	257.527812	3.098021	100.128746	2.303441
min	18.000000	-8019.000000	1.000000	0.000000	1.000000	-1.000000	0.000000
25%	33.000000	72.000000	8.000000	103.000000	1.000000	-1.000000	0.000000
50%	39.000000	448.000000	16.000000	180.000000	2.000000	-1.000000	0.000000
75%	48.000000	1428.000000	21.000000	319.000000	3.000000	-1.000000	0.000000
max	95.000000	102127.000000	31.000000	4918.000000	63.000000	871.000000	275.000000

Cell ini digunakan untuk menampilkan statistik deskriptif dari atribut numerik pada dataset. Informasi yang diperoleh meliputi jumlah data (count), rata-rata (mean), standar deviasi (std), nilai minimum (min), kuartil, median, dan nilai maksimum (max). Hasil statistik deskriptif membantu memahami rentang dan penyebaran nilai atribut numerik sebelum dilakukan analisis lebih lanjut.

3.2 Exploratory Data Analysis (EDA)

Cell 5 – Memeriksa Missing Values

```
In [5]: #202431111_RendyPradikaAbiyoga
        # NIM : 202431111
        # Nama : Rendy Pradika Abiyoga

        # Memeriksa jumlah missing values setiap kolom
        print("Missing Values Setiap Kolom:")
        print(df.isnull().sum())

        # Menghitung total missing values
        print("\nTotal Missing Values:", df.isnull().sum().sum())
```

```
Missing Values Setiap Kolom:
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64

Total Missing Values: 0
```

Cell ini digunakan untuk memeriksa keberadaan nilai kosong (missing values) pada setiap atribut dataset. Fungsi isnull().sum() menghitung jumlah nilai kosong pada masing-masing kolom, sedangkan penjumlahan keseluruhan digunakan untuk mengetahui total nilai kosong dalam dataset.

Berdasarkan hasil pemeriksaan, seluruh atribut memiliki nilai kosong sebanyak 0. Hal ini menunjukkan bahwa dataset Bank Customers tidak memiliki permasalahan missing values sehingga tidak diperlukan proses pengisian atau penghapusan data berdasarkan kondisi tersebut.

Cell 6 – Memeriksa Data Duplikat

```
In [6]: #202431111_RendyPradikaAbiyoga
        # NIM : 202431111
        # Nama : Rendy Pradika Abiyoga

        # Menghitung jumlah data duplikat
        print("Jumlah Data Duplikat:", df.duplicated().sum())
```

```
Jumlah Data Duplikat: 0
```

Cell ini digunakan untuk mengetahui apakah terdapat baris data yang memiliki isi identik dengan baris lainnya. Pemeriksaan dilakukan menggunakan fungsi duplicated().sum().

Berdasarkan hasil pemeriksaan, tidak ditemukan data duplikat pada dataset. Dengan demikian, seluruh 45.211 baris data dapat dipertahankan untuk tahapan analisis selanjutnya.

Cell 7 – Menganalisis Distribusi Target

```
In [7]: #202431111_RendyPradikaAbiyoga
        # NIM : 202431111
        # Nama : Rendy Pradika Abiyoga

        # Menghitung jumlah setiap kelas target
        print("Distribusi Target:")
        print(df['y'].value_counts())

        # Menghitung persentase setiap kelas target
        print("\nPersentase Target:")
        print(df['y'].value_counts(normalize=True) * 100)
```

```
Distribusi Target:
no     39922
yes     5289
Name: y, dtype: int64

Persentase Target:
no     88.30152
yes    11.69848
Name: y, dtype: float64
```

Cell ini digunakan untuk mengetahui distribusi jumlah nasabah berdasarkan atribut target y. Atribut tersebut menunjukkan apakah nasabah berlangganan deposito berjangka atau tidak.

Hasil analisis menunjukkan bahwa terdapat 39.922 nasabah dengan kategori no dan 5.289 nasabah dengan kategori yes. Persentase masing-masing kelas adalah sekitar 88,30% dan 11,70%. Kondisi ini menunjukkan bahwa dataset memiliki distribusi kelas yang tidak seimbang (class imbalance), karena jumlah nasabah yang tidak berlangganan deposito jauh lebih banyak dibandingkan nasabah yang berlangganan.

Cell 8 – Visualisasi Distribusi Target

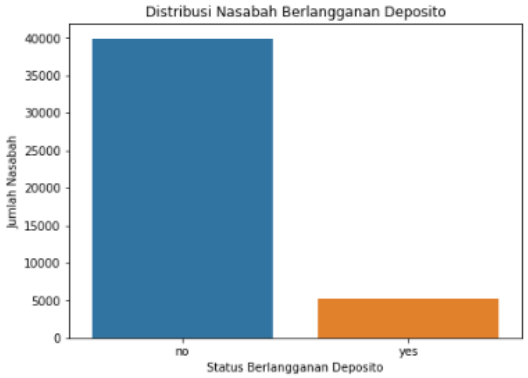
```
In [8]: #202431111_RendyPradikaAbiyoga
        # NIM : 202431111
        # Nama : Rendy Pradika Abiyoga

        # Membuat visualisasi distribusi target
        plt.figure(figsize=(7, 5))

        sns.countplot(data=df, x='y')

        plt.title('Distribusi Nasabah Berlangganan Deposito')
        plt.xlabel('Status Berlangganan Deposito')
        plt.ylabel('Jumlah Nasabah')

        plt.show()
```



Cell ini digunakan untuk memvisualisasikan distribusi kelas target menggunakan diagram batang (count plot). Grafik menunjukkan perbandingan jumlah nasabah yang berlangganan dan tidak berlangganan deposito berjangka.

Berdasarkan grafik, kategori no memiliki jumlah yang jauh lebih tinggi dibandingkan kategori yes. Visualisasi ini memperjelas adanya ketidakseimbangan kelas dalam dataset yang perlu diperhatikan ketika melakukan pelatihan dan evaluasi model klasifikasi.

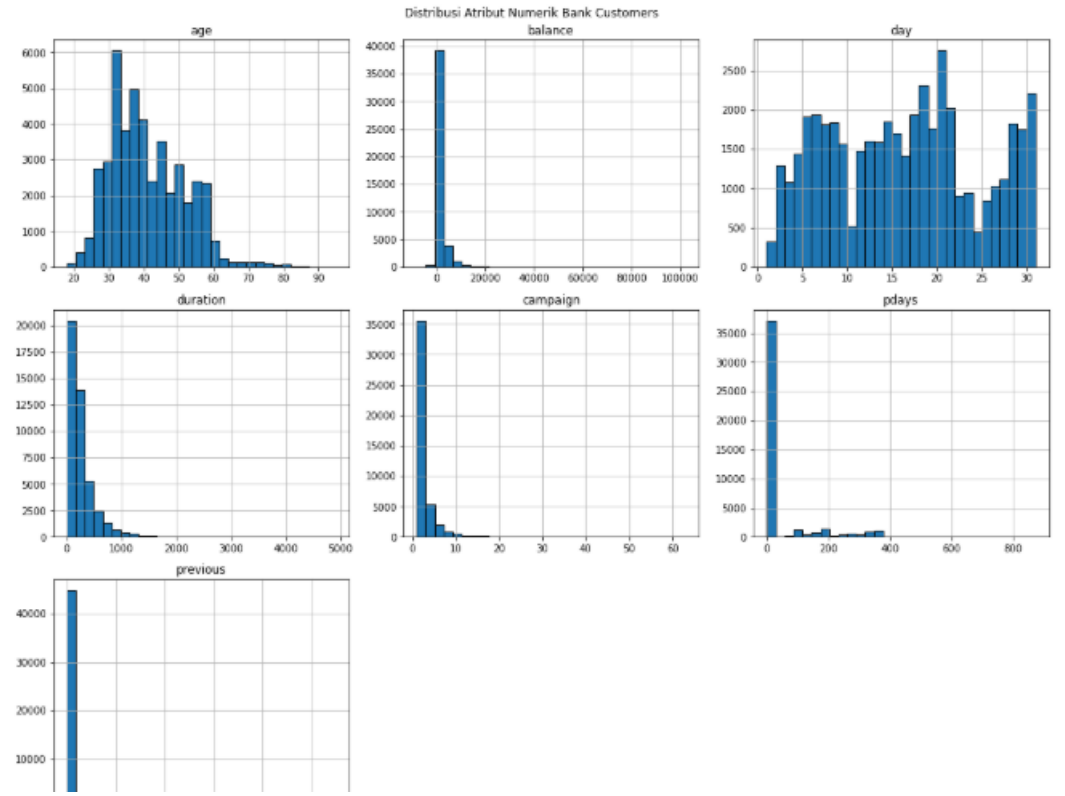
Cell 9 – Visualisasi Distribusi Atribut Numerik

```
In [9]: #202431111_RendyPradikaAbiyoga
        # NIM : 202431111
        # Nama : Rendy Pradika Abiyoga

        # Memilih atribut numerik
        numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns

        # Membuat histogram setiap atribut numerik
        df[numerical_columns].hist(
            figsize=(15, 12),
            bins=30,
            edgecolor='black'
        )

        plt.suptitle('Distribusi Atribut Numerik Bank Customers')
        plt.tight_layout()
        plt.show()
```



Cell ini digunakan untuk melihat distribusi nilai dari seluruh atribut numerik menggunakan histogram. Atribut yang divisualisasikan meliputi usia (age), saldo rekening (balance), hari kontak (day), durasi komunikasi (duration), jumlah kontak kampanye (campaign), jumlah hari sejak kontak sebelumnya (pdays), dan jumlah kontak sebelumnya (previous).

Histogram membantu mengidentifikasi pola penyebaran data, rentang nilai, serta kemungkinan adanya nilai ekstrem (outliers) pada atribut numerik.

Cell 10 – Analisis Korelasi Atribut Numerik

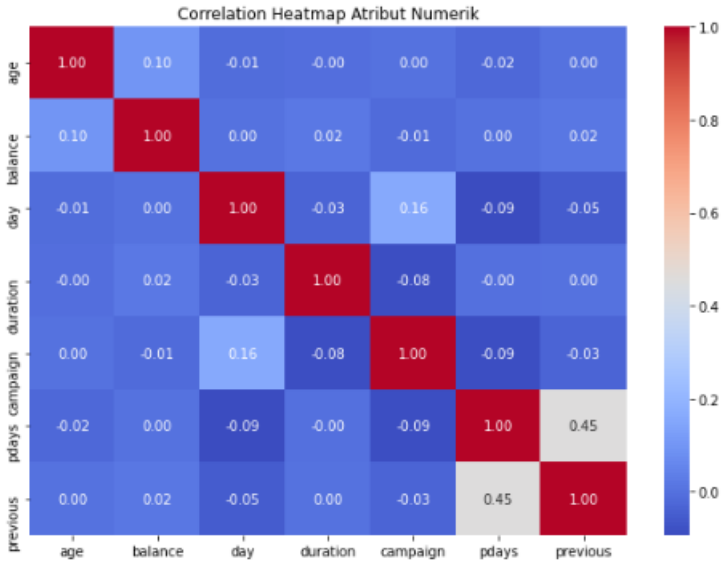
```
In [10]: #202431111_RendyPradikaAbiyoga
         # NIM : 202431111
         # Nama : Rendy Pradika Abiyoga

         # Mengambil atribut numerik
         numerical_df = df.select_dtypes(include=['int64', 'float64'])

         # Menghitung korelasi
         correlation = numerical_df.corr()

         # Membuat heatmap korelasi
         plt.figure(figsize=(10, 7))
         sns.heatmap(
             correlation,
             annot=True,
             cmap='coolwarm',
             fmt='.2f'
         )

         plt.title('Correlation Heatmap Atribut Numerik')
         plt.show()
```



Cell ini digunakan untuk menganalisis hubungan antaratribut numerik menggunakan metode korelasi Pearson. Hasil perhitungan ditampilkan dalam bentuk heatmap untuk mempermudah identifikasi kekuatan dan arah hubungan antarvariabel. Nilai korelasi mendekati 1 menunjukkan hubungan positif yang kuat, nilai mendekati -1 menunjukkan hubungan negatif yang kuat, sedangkan nilai mendekati 0 menunjukkan hubungan linear yang lemah.

Perlu diperhatikan bahwa korelasi tidak selalu menunjukkan hubungan sebab-akibat. Selain itu, analisis ini hanya mencakup atribut numerik dan belum menggambarkan hubungan atribut kategorikal dengan target.

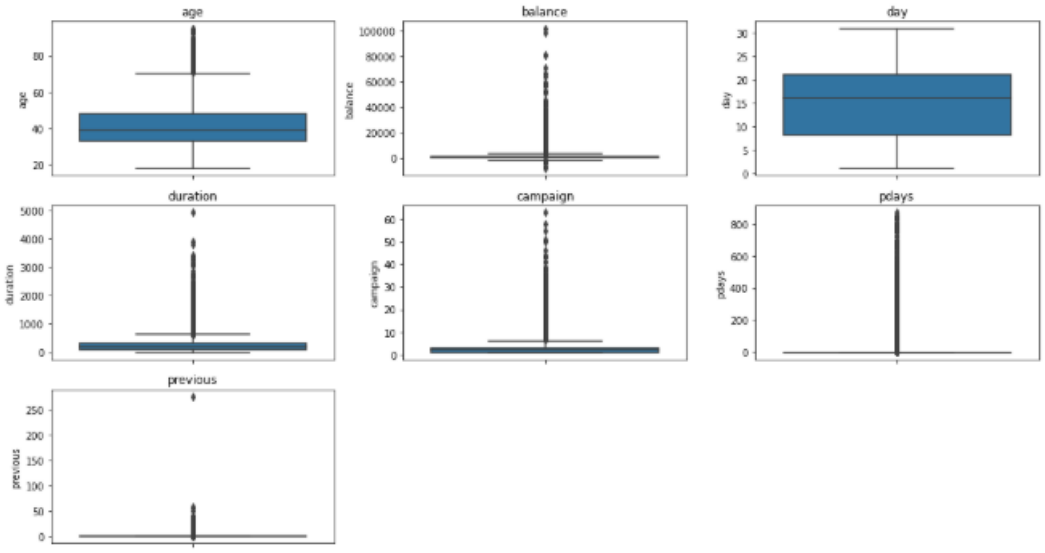
Cell 11 – Memeriksa Outliers

```
In [11]: #202431111_RendyPradikaAbiyoga
         # NIM : 202431111
         # Nama : Rendy Pradika Abiyoga

         # Membuat boxplot atribut numerik
         plt.figure(figsize=(15, 8))

         for i, column in enumerate(numerical_columns, 1):
             plt.subplot(3, 3, i)
             sns.boxplot(y=df[column])
             plt.title(column)

         plt.tight_layout()
         plt.show()
```



Cell ini digunakan untuk mengidentifikasi kemungkinan adanya nilai ekstrem pada atribut numerik melalui visualisasi boxplot. Boxplot menampilkan median, kuartil, dan titik data yang berada di luar batas whisker. Titik yang berada di luar whisker merupakan indikasi kandidat outliers, tetapi tidak selalu berarti data tersebut salah atau harus dihapus.

Pada dataset pelanggan bank, nilai seperti saldo rekening dan durasi komunikasi dapat memiliki rentang yang lebar. Oleh karena itu, nilai ekstrem perlu dianalisis terlebih dahulu sebelum menentukan tindakan preprocessing.

3.3 Data Preprocessing

Cell 12 – Menentukan Atribut Numerik dan Kategorikal

Cell ini digunakan untuk memisahkan atribut dataset berdasarkan tipe datanya, yaitu numerik dan kategorikal. Atribut numerik berisi nilai berbentuk angka, sedangkan atribut kategorikal berisi informasi berupa kelompok atau kategori tertentu. Atribut target y tidak dimasukkan ke dalam kelompok fitur karena digunakan sebagai variabel yang akan diprediksi oleh model.

Pemisahan ini diperlukan karena atribut numerik dan kategorikal membutuhkan metode pengolahan yang berbeda sebelum digunakan dalam proses klasifikasi.

In [12]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Memisahkan atribut numerik dan kategorikal
numerical_features = df.select_dtypes(
    include=['int64', 'float64']
).columns.tolist()

categorical_features = df.select_dtypes(
    include=['object']
).drop(columns=['y']).columns.tolist()

print("Atribut Numerik:")
print(numerical_features)

print("\nAtribut Kategorikal:")
print(categorical_features)

print("\nAtribut Target:")
print("y")
```

```
Atribut Numerik:
['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']

Atribut Kategorikal:
['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']

Atribut Target:
y
```

In [13]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Menghitung outliers menggunakan metode IQR
for column in numerical_features:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[
        (df[column] < lower_bound) |
        (df[column] > upper_bound)
    ]

    print(f"{column}: {len(outliers)} outliers")
```

```
age: 487 outliers
balance: 4729 outliers
day: 0 outliers
duration: 3235 outliers
campaign: 3064 outliers
pdays: 8257 outliers
previous: 8257 outliers
```

Cell ini digunakan untuk menghitung jumlah kandidat outliers pada setiap atribut numerik menggunakan metode Interquartile Range (IQR). Metode ini menentukan batas bawah dan batas atas berdasarkan kuartil pertama (Q1), kuartil ketiga (Q3), serta selisih keduanya.

Data yang berada di luar batas IQR akan teridentifikasi sebagai kandidat outliers. Namun, nilai tersebut tidak langsung dihapus karena dapat merepresentasikan kondisi nasabah yang sebenarnya, seperti saldo rekening yang tinggi atau durasi komunikasi yang panjang.

Catatan analisis: Atribut pdays memiliki nilai -1 yang dalam dataset ini menunjukkan nasabah belum pernah dihubungi pada kampanye sebelumnya. Nilai tersebut bukan missing value sehingga tidak boleh langsung dihapus atau diganti dengan median.

Cell 14 – Memisahkan Fitur dan Target

In [14]:

```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Memisahkan fitur dan target
X = df.drop(columns=['y'])
y = df['y']

print("Jumlah Fitur:", X.shape)
print("Jumlah Target:", y.shape)

print("\nContoh Fitur:")
display(X.head())

print("\nContoh Target:")
display(y.head())
```

```
Jumlah Fitur: (45211, 16)
Jumlah Target: (45211,)

Contoh Fitur:
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	p
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	

```
Contoh Target:

0    no
1    no
2    no
3    no
4    no
Name: y, dtype: object
```

Cell ini digunakan untuk memisahkan dataset menjadi variabel fitur (X) dan variabel target (y). Variabel X berisi 16 atribut karakteristik nasabah yang digunakan sebagai dasar prediksi, sedangkan variabel y berisi status nasabah dalam berlangganan deposito berjangka.

Pemisahan dilakukan agar algoritma dapat mempelajari hubungan antara karakteristik nasabah dan status berlangganan deposito.

Cell 15 – Membagi Data Training dan Testing

In [15]:

```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

from sklearn.model_selection import train_test_split

# Membagi data dengan perbandingan 80:20
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Jumlah Data Training:", X_train.shape[0])
print("Jumlah Data Testing:", X_test.shape[0])

print("\nDistribusi Target Training:")
print(y_train.value_counts())

print("\nDistribusi Target Testing:")
print(y_test.value_counts())
```

```
Jumlah Data Training: 36168
Jumlah Data Testing: 9043

Distribusi Target Training:
no     31937
yes     4231
Name: y, dtype: int64

Distribusi Target Testing:
no     7985
yes    1058
Name: y, dtype: int64
```

Cell ini digunakan untuk membagi dataset menjadi data pelatihan (training data) dan data pengujian (testing data) dengan perbandingan 80:20. Sebanyak 80% data digunakan untuk melatih model, sedangkan 20% sisanya digunakan untuk menguji kemampuan model dalam melakukan klasifikasi.

Parameter random_state=42 digunakan agar pembagian data dapat direproduksi, sedangkan stratify=y digunakan untuk mempertahankan proporsi kelas target pada data training dan testing. Penggunaan stratifikasi penting karena dataset memiliki distribusi kelas yang tidak seimbang.

Cell 16 – Encoding dan Normalisasi Data

In [16]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Membuat preprocessing untuk atribut numerik dan kategorikal
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ]
)

# Melakukan preprocessing pada data training dan testing
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("Bentuk Data Training Setelah Preprocessing:",
      X_train_processed.shape)

print("Bentuk Data Testing Setelah Preprocessing:",
      X_test_processed.shape)
```

```
Bentuk Data Training Setelah Preprocessing: (36168, 51)
Bentuk Data Testing Setelah Preprocessing: (9043, 51)
```

Cell ini digunakan untuk mengubah atribut kategorikal menjadi representasi numerik menggunakan metode One-Hot Encoding. Selain itu, atribut numerik diproses menggunakan StandardScaler untuk menyamakan skala data berdasarkan rata-rata dan standar deviasi.

Pengolahan dilakukan menggunakan ColumnTransformer sehingga masing-masing jenis atribut memperoleh metode preprocessing yang sesuai. Parameter preprocessing dipelajari melalui data training menggunakan fit_transform(), kemudian diterapkan pada data testing menggunakan transform().

Pemisahan tersebut dilakukan untuk menghindari data leakage, yaitu kondisi ketika informasi dari data pengujian ikut digunakan dalam proses pembelajaran model.

3.4 Pemodelan menggunakan Naive Bayes

Cell 17 – Import library dan membuat model

In [17]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

from sklearn.naive_bayes import GaussianNB

# Membuat model Gaussian Naive Bayes
model_nb = GaussianNB()

# Melatih model menggunakan data training
model_nb.fit(X_train_processed, y_train)

print("Model Gaussian Naive Bayes berhasil dilatih.")
```

```
Model Gaussian Naive Bayes berhasil dilatih.
```

Pada tahap ini dilakukan pembangunan model klasifikasi menggunakan algoritma Gaussian Naive Bayes. Algoritma ini menerapkan Teorema Bayes dengan asumsi bahwa setiap atribut memiliki distribusi Gaussian atau normal pada masing-masing kelas. Model dilatih menggunakan data training yang telah melalui proses encoding dan scaling agar dapat digunakan dalam proses klasifikasi.

Cell 18 – Melakukan prediksi

In [18]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Melakukan prediksi terhadap data testing
y_pred_nb = model_nb.predict(X_test_processed)

print("Hasil prediksi Naive Bayes:")
print(y_pred_nb[:20])
```

```
Hasil prediksi Naive Bayes:
['no' 'no' 'no' 'no' 'no' 'no' 'no' 'yes' 'no' 'no' 'no' 'no' 'no' 'no'
 'no' 'no' 'no' 'no' 'yes' 'no']
```

Setelah proses pelatihan selesai, model digunakan untuk memprediksi data testing. Hasil prediksi menunjukkan kelas yang diperkirakan oleh model, yaitu apakah nasabah akan berlangganan produk deposito (yes) atau tidak (no). Prediksi ini selanjutnya dibandingkan dengan label sebenarnya untuk mengetahui performa model.

3.5 Pemodelan menggunakan K-Nearest Neighbors (KNN)

Cell 19 – Import library dan membuat model KNN

In [19]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

from sklearn.neighbors import KNeighborsClassifier

# Membuat model KNN dengan 5 tetangga terdekat
model_knn = KNeighborsClassifier(n_neighbors=5)

# Melatih model
model_knn.fit(X_train_processed, y_train)

print("Model KNN berhasil dilatih.")
```

```
Model KNN berhasil dilatih.
```

Pada tahap ini dilakukan pembangunan model klasifikasi menggunakan algoritma K-Nearest Neighbors dengan nilai \(K=5\). Artinya, model akan mempertimbangkan lima data training yang memiliki jarak terdekat dengan data yang akan diprediksi. Penentuan kelas dilakukan berdasarkan mayoritas kelas dari kelima tetangga tersebut. Standardisasi pada tahap preprocessing diperlukan karena KNN menggunakan perhitungan jarak sehingga perbedaan skala atribut dapat memengaruhi hasil klasifikasi.

Cell 20 – Melakukan prediksi KNN

In [20]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Melakukan prediksi terhadap data testing
y_pred_knn = model_knn.predict(X_test_processed)

print("Hasil prediksi KNN:")
print(y_pred_knn[:20])
```

```
Hasil prediksi KNN:
['no' 'no' 'no' 'no' 'no' 'no' 'no' 'yes' 'no' 'no' 'no' 'no' 'no' 'no'
 'no' 'no' 'no' 'no' 'no' 'no']
```

Model KNN yang telah dilatih digunakan untuk memprediksi kelas pada data testing. Hasil prediksi menunjukkan kategori nasabah yang diperkirakan oleh model berdasarkan lima tetangga terdekat. Hasil tersebut kemudian akan dievaluasi menggunakan metrik klasifikasi.

3.6 Evaluasi model

Cell 21 – Evaluasi Naive Bayes

In [21]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Menghitung akurasi
accuracy_nb = accuracy_score(y_test, y_pred_nb)

print("Akurasi Naive Bayes:", accuracy_nb)
print("\nClassification Report Naive Bayes:")
print(classification_report(y_test, y_pred_nb))

print("Confusion Matrix Naive Bayes:")
print(confusion_matrix(y_test, y_pred_nb))
```

```
Akurasi Naive Bayes: 0.8548048214088245

Classification Report Naive Bayes:
              precision    recall  f1-score   support

          no       0.93      0.90      0.92      7985
         yes       0.41      0.52      0.46      1058

    accuracy                           0.85      9043
   macro avg       0.67      0.71      0.69      9043
weighted avg       0.87      0.85      0.86      9043

Confusion Matrix Naive Bayes:
[[7180  805]
 [ 508  550]]
```

Evaluasi model Gaussian Naive Bayes dilakukan dengan membandingkan hasil prediksi terhadap label sebenarnya pada data testing. Accuracy menunjukkan proporsi prediksi yang benar dari seluruh data. Precision menunjukkan ketepatan prediksi pada suatu kelas,

recall menunjukkan kemampuan model menemukan data dari kelas tersebut, sedangkan F1-score merupakan rata-rata harmonis precision dan recall. Confusion matrix digunakan untuk melihat jumlah prediksi benar dan salah pada setiap kelas.

Cell 22 – Evaluasi KNN

In [22]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

# Menghitung akurasi
accuracy_knn = accuracy_score(y_test, y_pred_knn)

print("Akurasi KNN:", accuracy_knn)
print("\nClassification Report KNN:")
print(classification_report(y_test, y_pred_knn))

print("Confusion Matrix KNN:")
print(confusion_matrix(y_test, y_pred_knn))
```

```
Akurasi KNN: 0.8961627778392126

Classification Report KNN:
              precision    recall  f1-score   support

          no       0.92      0.97      0.94      7985
         yes       0.60      0.34      0.43      1058

    accuracy                           0.90      9043
   macro avg       0.76      0.66      0.69      9043
weighted avg       0.88      0.90      0.88      9043

Confusion Matrix KNN:
[[7744  241]
 [ 698  360]]
```

Evaluasi KNN dilakukan menggunakan metrik yang sama dengan Gaussian Naive Bayes agar hasil kedua algoritma dapat dibandingkan secara adil. Nilai accuracy, precision, recall, dan F1-score digunakan untuk mengetahui kemampuan model dalam mengklasifikasikan nasabah ke dalam kelas yes dan no.

Cell 23 – Membuat tabel perbandingan

In [23]:
```
#202431111_RendyPradikaAbiyoga
# NIM : 202431111
# Nama : Rendy Pradika Abiyoga

from sklearn.metrics import precision_score, recall_score, f1_score

# Membuat tabel perbandingan
hasil_perbandingan = pd.DataFrame({
    'Algoritma': ['Gaussian Naive Bayes', 'K-Nearest Neighbors'],
    'Accuracy': [
        accuracy_score(y_test, y_pred_nb),
        accuracy_score(y_test, y_pred_knn)
    ],
    'Precision (Yes)': [
        precision_score(y_test, y_pred_nb, pos_label='yes', zero_division=0),
        precision_score(y_test, y_pred_knn, pos_label='yes', zero_division=0)
    ],
    'Recall (Yes)': [
        recall_score(y_test, y_pred_nb, pos_label='yes', zero_division=0),
        recall_score(y_test, y_pred_knn, pos_label='yes', zero_division=0)
    ],
    'F1-Score (Yes)': [
        f1_score(y_test, y_pred_nb, pos_label='yes', zero_division=0),
        f1_score(y_test, y_pred_knn, pos_label='yes', zero_division=0)
    ]
})

hasil_perbandingan
```

Out[23]:

	Algoritma	Accuracy	Precision (Yes)	Recall (Yes)	F1-Score (Yes)
0	Gaussian Naive Bayes	0.854805	0.405904	0.519849	0.455864
1	K-Nearest Neighbors	0.896163	0.599002	0.340265	0.433996

Tabel perbandingan digunakan untuk melihat perbedaan performa Gaussian Naive Bayes dan K-Nearest Neighbors berdasarkan data testing yang sama. Selain accuracy, metrik precision, recall, dan F1-score pada kelas yes turut diperhatikan karena tujuan klasifikasi adalah mengidentifikasi nasabah yang berpotensi berlangganan deposito. Algoritma dengan nilai metrik yang lebih baik dapat dianggap memiliki performa lebih baik pada aspek yang diukur.

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan landasan teori dan hasil praktikum penerapan algoritma Gaussian Naive Bayes dan K-Nearest Neighbors (KNN) pada dataset Bank Customers, dapat disimpulkan bahwa:

1. Penerapan algoritma Machine Learning dapat digunakan untuk melakukan klasifikasi data nasabah berdasarkan karakteristik yang tersedia. Dalam praktikum ini, klasifikasi dilakukan untuk memprediksi apakah nasabah akan berlangganan produk atau layanan bank (yes) atau tidak (no).

2. Tahap preprocessing data memiliki peran penting sebelum proses klasifikasi dilakukan. Dataset yang digunakan terdiri dari 45.211 baris dan 17 kolom, dengan atribut numerik dan kategorikal. Data diproses menggunakan StandardScaler untuk menstandarkan atribut numerik dan OneHotEncoder untuk mengubah atribut kategorikal menjadi bentuk numerik agar dapat digunakan oleh algoritma Machine Learning. Pembagian data latih dan data uji dilakukan dengan perbandingan 80:20.

3. Algoritma Gaussian Naive Bayes memperoleh akurasi sebesar 85,48%. Pada kelas nasabah yang berlangganan (yes), algoritma ini menghasilkan precision sebesar 40,59%, recall sebesar 51,98%, dan F1-score sebesar 45,59%. Hasil tersebut menunjukkan bahwa Naive Bayes memiliki kemampuan yang lebih baik dalam menemukan nasabah yang benar-benar berlangganan dibandingkan KNN, berdasarkan nilai recall kelas yes.

4. Algoritma K-Nearest Neighbors (KNN) dengan parameter n_neighbors=5 memperoleh akurasi sebesar 89,62%. Pada kelas yes, KNN menghasilkan precision sebesar 59,90%, recall sebesar 34,03%, dan F1-score sebesar 43,40%. Hasil tersebut menunjukkan bahwa KNN memiliki akurasi keseluruhan dan precision kelas yes yang lebih tinggi dibandingkan Naive Bayes, tetapi masih kurang optimal dalam menemukan seluruh nasabah yang benar-benar berlangganan.

5. Perbandingan kedua algoritma menunjukkan bahwa KNN unggul dalam akurasi keseluruhan, sedangkan Gaussian Naive Bayes unggul dalam recall dan F1-score untuk kelas yes. Perbedaan hasil tersebut menunjukkan bahwa pemilihan algoritma terbaik perlu disesuaikan dengan tujuan klasifikasi. Jika tujuan utamanya adalah memperoleh prediksi keseluruhan yang lebih akurat, KNN dapat menjadi pilihan. Namun, jika tujuan utamanya adalah menemukan sebanyak mungkin nasabah yang berpotensi berlangganan, Gaussian Naive Bayes lebih sesuai berdasarkan hasil praktikum ini.

6. Ketidakseimbangan distribusi kelas pada dataset menjadi salah satu faktor yang memengaruhi hasil klasifikasi. Jumlah data nasabah yang tidak berlangganan (no) jauh lebih banyak dibandingkan nasabah yang berlangganan (yes). Oleh karena itu, penilaian performa model tidak cukup hanya menggunakan akurasi, tetapi juga perlu memperhatikan precision, recall, F1-score, dan confusion matrix.

Secara keseluruhan, praktikum ini menunjukkan bahwa algoritma Gaussian Naive Bayes dan KNN dapat diterapkan untuk melakukan klasifikasi pada dataset nasabah bank. Kedua algoritma memiliki kelebihan dan kekurangan masing-masing. Berdasarkan hasil pengujian, KNN memperoleh akurasi tertinggi sebesar 89,62%, sedangkan Gaussian Naive Bayes lebih baik dalam mendeteksi kelas nasabah yang berlangganan. Hasil ini menegaskan bahwa pemilihan model Machine Learning harus mempertimbangkan kebutuhan analisis dan karakteristik data yang digunakan.

DAFTAR PUSTAKA

Alenazi, F. S., El Hindi, K., & AsSadhan, B. (2023). Complement-class harmonized Naïve Bayes classifier. Applied Sciences, 13(8), 4852. https://doi.org/10.3390/app13084852

Alshdaifat, E., Alshdaifat, D., Alsarhan, A., Hussein, F., & El-Salhi, S. M. F. S. (2021). The effect of preprocessing techniques, applied to numeric features, on classification algorithms' performance. Data, 6(2), 11. https://doi.org/10.3390/data6020011

Anggraeni, L. D., & Kustiyono. (2026). Analisis komparatif Naive Bayes dan K-Nearest Neighbor dalam klasifikasi kinerja produk marketplace berbasis data mining dengan performa optimal pada Naive Bayes. Jurnal Algoritma, 23(1), 528–537. https://doi.org/10.33364/algoritma/v.23-1.3300

Cabot, J. H., & Ross, E. G. (2023). Evaluating prediction model performance. Surgery, 174(3), 723–726. https://doi.org/10.1016/j.surg.2023.05.023

Izonin, I., Tkachenko, R., Shakhovska, N., Ilchyshyn, B., & Singh, K. K. (2022). A two-step data normalization approach for improving classification accuracy in the medical diagnosis domain. Mathematics, 10(11), 1942. https://doi.org/10.3390/math10111942

Palanivinayagam, A., & Damaševičius, R. (2023). Effective handling of missing values in datasets for classification using machine learning methods. Information, 14(2), 92. https://doi.org/10.3390/info14020092

Wang, W., Yan, L., Liu, F., & Li, Y. (2025). Improving Gaussian Naive Bayes classification on imbalanced data through coordinate-based minority feature mining. PeerJ Computer Science, 11, e3003. https://doi.org/10.7717/peerj-cs.3003

Zhang, X., Xiao, H., Gao, R., Zhang, H., & Wang, Y. (2022). K-nearest neighbors rule combining prototype selection and local feature weighting for classification. Knowledge-Based Systems, 243, 108451. https://doi.org/10.1016/j.knosys.2022.108451